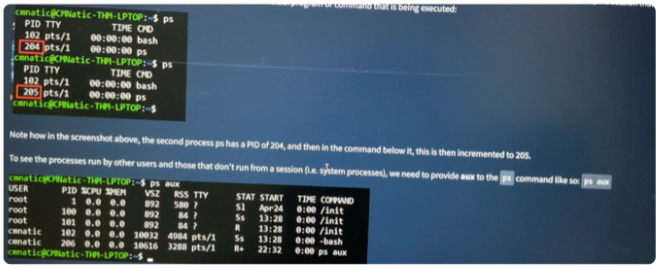
## Tasks :- processes 101

processes are programs that are running on your machine. They are managed by kernal. where each process will have an ID associated with it, also known as its PID. PID increments for order in which process starts i.e 60th process will have PID of 60.

### Viewing processes :-

we can use friendly ps command to provide list of running processes as an user's session and some additional information such as it's status code, the session that is running it, how much usage time of CPU it is using, and name of actual program or command that is being executed.




Note we can see total of 5 processes - note how we now have "root" and "Cmnatic".

* Another very useful command is top command; top gives you real time statistics about process running on your system instead of one time view. These statistics will refresh every 10 seconds, but will also refresh when you use the arrow key to browse various rows. Another great command to gain insight into your system is via top command.

```
top - 22:36:17 up 1 day,  6:32,  0 users,  load average: 0.00, 0.00, 0.00
Tasks:   5 total,   1 running,   4 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.8 sy,  0.0 ni, 99.2 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  12630.9 total,  12206.5 free,     83.6 used,    340.9 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.  12306.1 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root      20   0     892    580    516 S   0.0   0.0   0:00.93 init
  100 root      20   0     892     84     20 S   0.0   0.0   0:00.00 init
  101 root      20   0     892     84     20 S   0.0   0.0   0:00.07 init
  102 cmnatic   20   0   10032   4988   3272 S   0.0   0.0   0:00.08 bash
  209 cmnatic   20   0   10872   3704   3188 R   0.0   0.0   0:00.00 top
```

### Managing processes :-

you can send signals that terminate processes : there are variety types of signals that

correlate to exactly how "cleanly" process is dealt with by kernal. To kill a command, we can use appropriately named kill command and associated PID to wish to kill. kill PID 1337, we'd use kill 1337.

Below are some of signals that we can send to processes when it is killed.

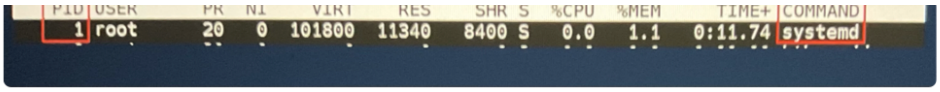
* SIGTERM - Kill the process, but allow it to do some cleanup tasks before hand.
* SIGKILL :- Kill the process, doesn't do any cleanup after the fact.
* SIGSTOP :- STOP/Suspend a process.

## How do processes Start?

lets start off by talking about namespaces. The Operating System (OS) uses namespaces to ultimately split up the resources available on Computer to (such as CPU, RAM and disk) processes. Think of it as splitting your computer up into slices - similar to cake. Processes within that slice will have ques to certain amount of computing power, however it will have small portion of what is actually available to every process overall.

* Namespaces are great for security as it is way of isolating processes from another.
* Previously we talked about PID how it works. The process with an ID of 0 is process that is started when System boots. This process in Systems init on Ubuntu, such as Systemd, which is used to provide way of managing user's processes and sits b/w operating System and user.

Ex:- once a System boots and initialises, Systemd is one of first processes that are started. Any program & piece of software that we want to start will start as what's known as child process of Systemd. This means that is controlled by Systemd, but will run as its own process to make it easier for us identify like.



| PID | USER | PR | NI | VIRT | RES | SHR | S | %CPU | %MEM | TIME+ | COMMAND |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | root | 20 | 0 | 101800 | 11340 | 8400 | S | 0.0 | 1.1 | 0:11.74 | systemd |

## Getting processes / services to start on Boot :-

Some applications can be started on boot of system that we own. Ex:- Web servers, database servers or file transfer servers. This software is often critical and often told to start during the boot up off system by administrators.

Ex:- If you are going to be telling apache web server to be starting apache manually and then telling the system to launch apache d on boot.

**Systemctl** - This command allows us to interact with **systemd** process/daemon. Systemctl is easy to use command that takes the following formatting:- Systemctl [option] [service]

Ex:- To tell apache to start up, we'll use **Systemctl start apache 2**, Same if you want to **stop** apache, we'd just replace the [option] with Stop (instead of start like we provide).

We can do 5 options with **Systemctl**:

* Start * Stop * Enable * Disable * Status

## An introduction to Backgrounding and Foregrounding in Linux :-

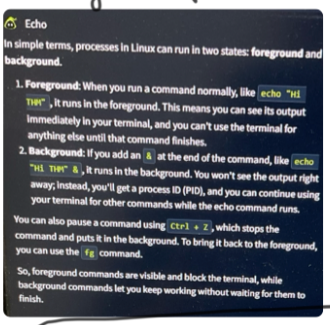
processes can run in 2 states :- In the background and in the foreground.

Ex:- Commands that you run in your terminal such as "echo" or things of that sort will run in foreground of your terminal as it is the only command provided that hasn't been told to run in background. Ex:- Echo will return to you in foreground, but wouldn't in background.

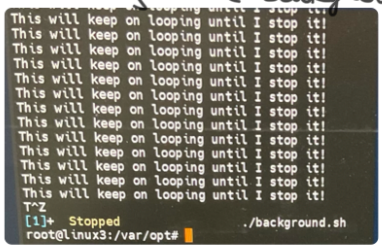
```
root@linux3:~# echo "Hi THM"
Hi THM
root@linux3:~# echo "Hi THM" &
[1] 16889
root@linux3:~# Hi THM
```

Here, we are running echo "Hi THM", where we expect the output returned to us like it is at

the start. But after adding & operator to command, were instead just given the ID of echo process rather than actual output -- as it is running in the background.



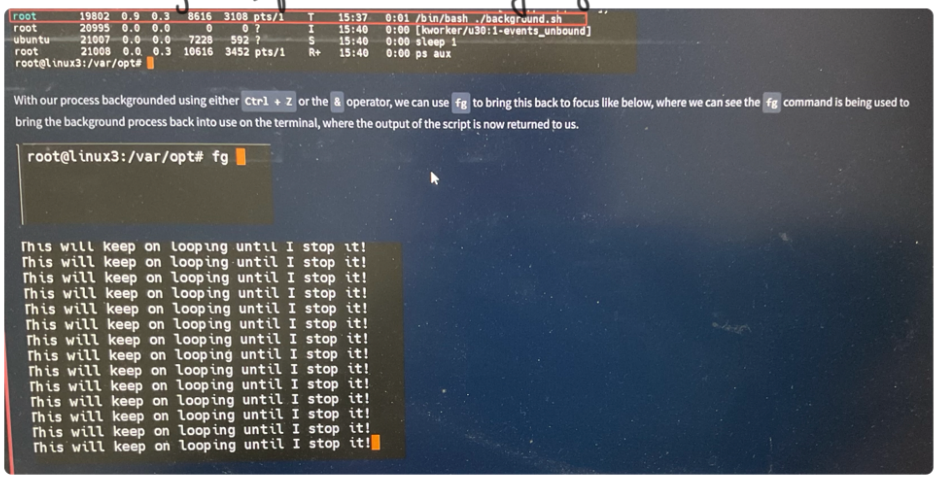

Effective way of "pausing" the execution of script or command like example in right side ->




↳ The Script will keep on repeating "This will keep on looping until i stop" until i stop or suspend the process • By using Ctrl+Z. Now our terminal is no longer filled up with messages -- until we foreground it, which we discuss below.

Foregrounding a process :-

Now that we have process running in background. For example, our Script "background.sh" which is confirmed by using ps aux command, we can bank pedal and bring this process back to foreground to interact with.




Quiz :-

1) If we were to launch a process where previous ID was "300", what would be ID of new process be?

A) 301

2) If we wanted to cleanly kill a process, what Signal would we send it?

A) SIGTERM

3) what command would we use to stop the service "myService"?

A) Systemctl stop myService

4) what command would we use to start the same service on boot up System?

A) Systemctl Enable myService

5) what command would we use to bring previously backgrounded process back to foreground?

A) fg